Chapter 7: Normalized Database Design Part 2

Abu Raihan Mostofa Kamal

Professor, CSE Department
Islamic University of Technology (IUT)

November 6, 2023

[1]This is based on Textbook, its companion slide and other sources

Chapter Outline

Closure of Attributes → set of given + applied Func. Dependency.

Normalization

Fist Normal Form and Atomic Domain

Second Normal Form

Third Normal Form

Boyce - Codd Normal Form (BCNF)



Closure of Attribute Sets: X+

SID → Name, CGPA, Hall,

Attribute Closure X+

Attribute closure **X+** of an attribute set **X** can be defined as set of attributes which can be functionally determined from it.

Purpose:

- **Given a Relation and a set of FD**s, it is **used to determine** if a given attribute set is a **Candidate Key** or not.
- Determination of **all possible Candidate Keys** are needed for **testing of almost all Normal Forms** (will be discussed later).

Keys (Revisited) in terms of X+

Superkey

A superkey is a set of attribute (X) whose closure(X+) contains **all the attributes** of the given relation.

This definition is exactly identical to our previous definition: it can uniquely identify **each record**, but here it is presented in terms of attribute closure.

Candidate Key

A candidate key is a superkey whose **proper subset** is **not a superkey**. Proper subset means: **at least one element less.** (i.e. i.e.There shall not be any other superkey whose length is less than the present one)

This definition is exactly identical to our previous definition: it is the **minimum superkey**, but here it is presented in terms of attribute closure.

Closure of Attribute Sets(X+): How to compute (Meth1)

Given Relation $R(A, B, C, D, E)$ and **FD:** $A \longrightarrow B$, $B \longrightarrow C$,$C \longrightarrow D$,$D \longrightarrow E$

Method 1: Using Armstrong's Axioms without any optimization. **Simple trick:** Compute all $\alpha \longrightarrow \beta_i$ for all possible i. And finally **union** them to get the α^+

We can reduce:

- $A \longrightarrow C$ (transitivity)
- $A \longrightarrow A$ (reflexivity)
- $A \longrightarrow D$ (transitivity)
- $A \longrightarrow E$ (transitivity)
- $A \longrightarrow A, B, C, D, E$ (union)
- So, $A^+ = \{A, , B, C, D, E\}$

- $B \longrightarrow D$ (tran)
- $B \longrightarrow E$ (tran)
- $B \longrightarrow B$
- $B \longrightarrow B, C, D, E$ (U.)
- So, $B^+ = \{B, C, D, E\}$

- $C \longrightarrow E$
- $C \longrightarrow C$
- $C \longrightarrow C, D, E$
- So, $C^+ = \{C, D, E\}$

Process goes like that...**But** it is a long, time-consuming and very naive approach specially for a larger set of FDs and larger number of attributes.

Closure of Attribute Sets(X+): How to compute (Method 2)

Given Relation **R(A,B,C,D,E)** and **FD:** $A \longrightarrow B$, $B \longrightarrow C$,$C \longrightarrow D$,$D \longrightarrow E$

Method 2: Simplified approach

Basic Principle

First use **Reflexivity Rule** for a given attribute set, then iterate through the **given FD**s to add element in the set.When you finish the last FD, re-iterate if new element can be added. **Simple trick:** if $\alpha \longrightarrow \beta$ holds, and α is already in the Set, then add β (as α can determine β).



Closure of Attribute Sets(X+): How to compute (Method 2)

Given Relation **R(A,B,C,D,E)** and **FD:** $A \longrightarrow B, B \longrightarrow C, C \longrightarrow D, D \longrightarrow E$

Method 2: Simplified approach. We will compute **AD+** first:

- $AD^+ = \{A, D, B, E, C\}$
- $AB^+ = \{A, B, C, D, E\}$
- $B^+ = \{B, C, D, E\}$
- $CD^+ = \{C, D, E\}$
- $C^+ = \{C, D, E\}$

Determination of Superkey based on X+

(Recall the definition:) A superkey is a set of attribute (X) whose closure(X+) contains **all the attributes** of the given relation.
So far we have computed X+ as follows:

- $A^+ = \{A, B, C, D, E\}$ Superkey ✔
- $AD^+ = \{A, B, C, D, E\}$ Superkey ✔
- $B^+ = \{B, C, D, E\}$ Superkey ✘(A is absent)
- $AB^+ = \{A, B, C, D, E\}$ Superkey ✔
- $C^+ = \{C, D, E\}$ Superkey ✘ (A,B are absent)
- $CD^+ = \{C, D, E\}$ Superkey ✘ (A,B are absent)

Determination of Candidate Key

(Recall the definition:) A candidate key is a superkey(SK) whose **proper subset** is **not a superkey**. Proper subset means: at least one element less.
So far we have computed X+ as follows:

- $A^+ = \{A, B, C, D, E\}$ Superkey ✔ Candidate key ✔
- $AD^+ = \{A, B, C, D, E\}$ Superkey ✔ Not a Candidate key ✘(A is a proper subset of AD and A is already a SK)
- $B^+ = \{B, C, D, E\}$ Superkey ✘(A is absent)
- $AB^+ = \{A, B, C, D, E\}$ Superkey ✔ Not a Candidate key ✘(A is a proper subset of AD and A is already a SK)
- $C^+ = \{C, D, E\}$ Superkey ✘ (A,B are absent)
- $CD^+ = \{C, D, E\}$ Superkey ✘ (A,B are absent)

So, A is the only Candiate Key

Recap of Method 1 and Method 2

Method 1 : Just use all possible A. Axioms to determine X^+.

But, it is hard to compute for a large X.

Method 2 : Axioms with a simple technique.

Basic Principle : For a given X

i. First use Reflexivity Rule

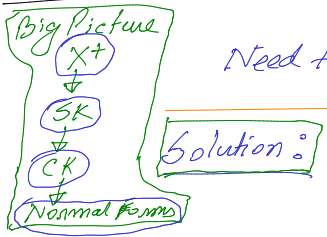
ii. Look at the given FDs to include elements (if possible)

Recap of Method 1 and Method 2 (Cont.)

Example (same) : Given $R(A, B, C, D, E)$

FD : $A \to B$, $B \to C$, $C \to D$, $D \to E$

Need to compute : X^+ (X is a set of attributes)



Solution :

$A^+ = \{A\}$ Ref. Rule (RR)

$= A, B$ (FD : $A \to B$)

$= A, B, C$ (FD : $B \to C$)

$= A, B, C, D$ (FD : $C \to D$)

$= A, B, C, D, E$ (FD : $D \to E$)

$= \{A, B, C, D, E\}$ (all attributes)

So, A is a Superkey (SK)

Recap of Method 1 and Method 2 (Cont.2)

Method 2 : Another Example Given : $R(A,B,C,D,E)$

$FD = \{A \to B,\ D \to E\}$

Solution : Lets start with A

SK = Superkey

CK = Candidate Key

So, $A^+ = \{A\}$ (RR)

$= \{A, B\}$ FD $A \to B$

Thus, $BC^+ = \{B, C\}$

[$\alpha \to \beta$ means α can determine β]

Now Lets check,

1. $ABCDE^+ = \{A, B, C, D, E\}$ (so is a SK) ✓
2. $ABDE^+ = \{A, B, D, E\}$ only RR, no FD helps here (NOT SK)
3. $ACDE^+ = \{A, C, D, E, B\}$ RR + FD : $A \to B$ (SK) ✓
4. $ACD^+ = \{A, C, D, B, E\}$ RR + FD : $A \to B$, $D \to E$ (SK) (ACD) ✓

Method 1 and Method 2: Problem

Both methods are **hard to implement** in real perspective especially when the R and FDs are large as the methods basically follow the brute-force approach. We are in need of an **optimized method**. We term it **Method 3**. In practice Method 3 is used.

Acronym used:
SK : Superkey
CK : Candidate Key
PSS : Proper Subset
RS : Reduced Set
PA : Prime Attributes
DA : Dependent Attributes
RP : Replacement Process



Method 3: Algorithm Overview (High-level)

Input: A relation **R(A,B,C...)** and a set of FDs: $A \longrightarrow B,C \longrightarrow D$...
Output: Possible sets of Candidate Keys
Step 1: Compute **Closure of All** Attributes+
then **discard** using given and implied FDs to get a Reduced Set (RS)
Now, check all PSS of the RS, if No PSS is a SK
then RS is the CK

Step 2: (Check for more CKs)
Form **Dependent Attributes (DA)** (static) from the given FDs
and **Prime Attribute (PA)** (all attributes of the CK discovered so far) (dynamic)
Now Compute $\alpha = DA \cap PA$
if α is NULL (empty) then no other CK is possible
else go for **Replacement Process (RP)** for each element of α
For each RP go to Step 1 to check if it a CK.

Method 3

Method 3 Basic Principle –

Step 1 : Start with all attributes and then (i) eliminate attribute using FDs

e.g. If ABC in (i) and FD: B → C

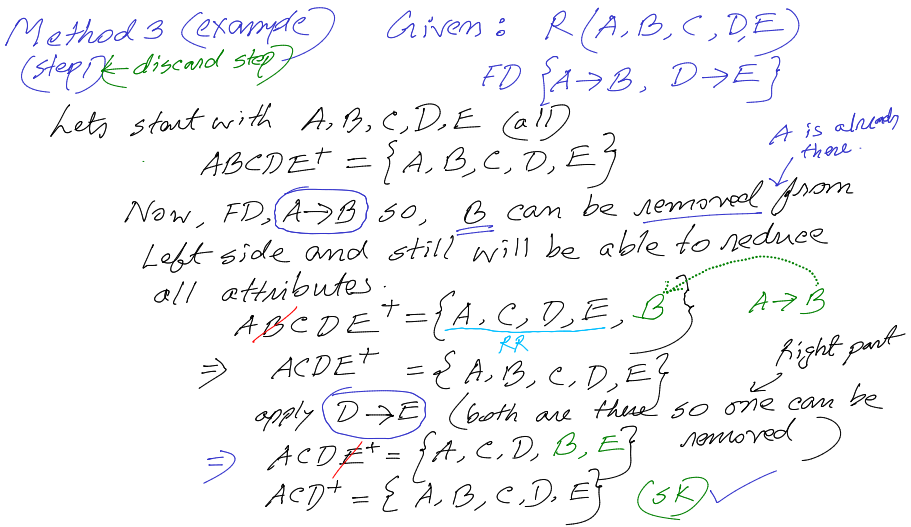
Here C can be removed as B can determine C and B is already in the set.

AB ⇏ AB

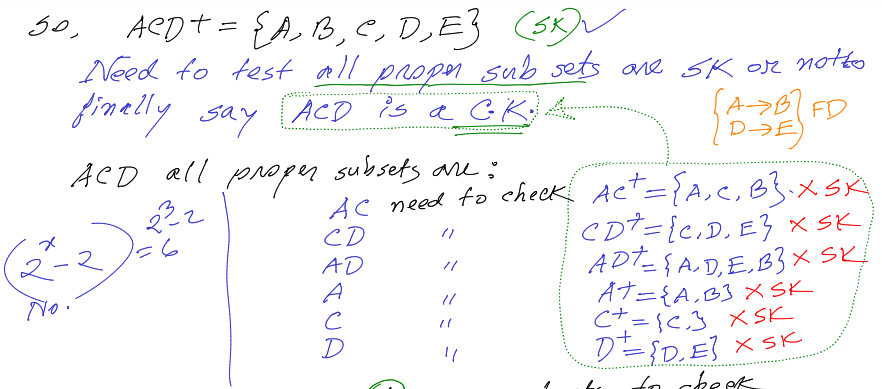
{other step will be discussed once we finish this step by example}

Benifit of step 1 : No of proper subsets will be significantly reduced.

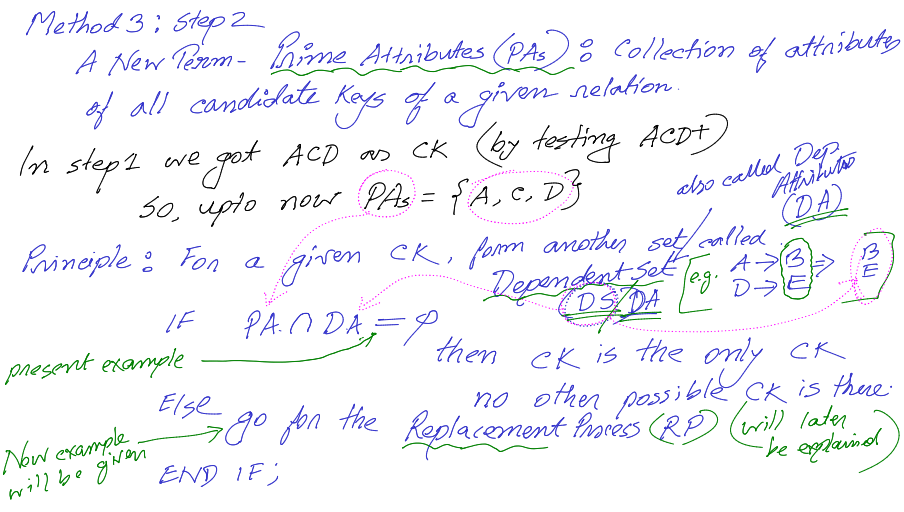
Method 3: Simple Example



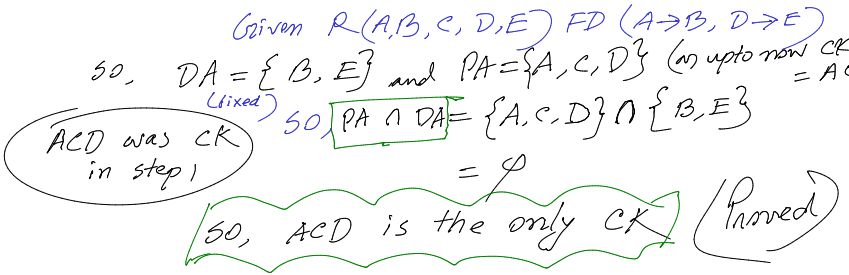
Method 3: Simple Example Cont.



Method 3: Simple Example Cont.



Method 3: Simple Example (End)



Method 3: Standard Example (Begin)

New Example for the case $CKi \cap DS \neq \phi$

Given: $R(A,B,C,D)$ & FD: $A \to B$, $B \to C$, $C \to A$

Step1 $ABCD^+ = \{A, B, C, D\}$

discard use: $A \to B \Rightarrow ABCD$ (since both A, B are there and B can be deduced from A so, B is redundant)

discard on FR: $A \to B$ & $B \to C \Rightarrow A \to C$

$ACD = ACD$, so, we get AD

No more discard possible.

So, $AD^+ = A, B, C, D$ [we dont need to check now as discard method ensures it]

Now check all proper subsets of AD (i.e A, D)

Method 3: Standard Example Cont.

$A^+ = A, B, C$ NOT a SK

$D^+ = D$ NOT a SK

FD $A \to B$ $B \to C$ $C \to A$

So, AD is a candidate Key (CK).

New step: Now we need to check if some CK exist.

Upto now – $PA_s = A, D$

(fixed) $DA = B, C, A$

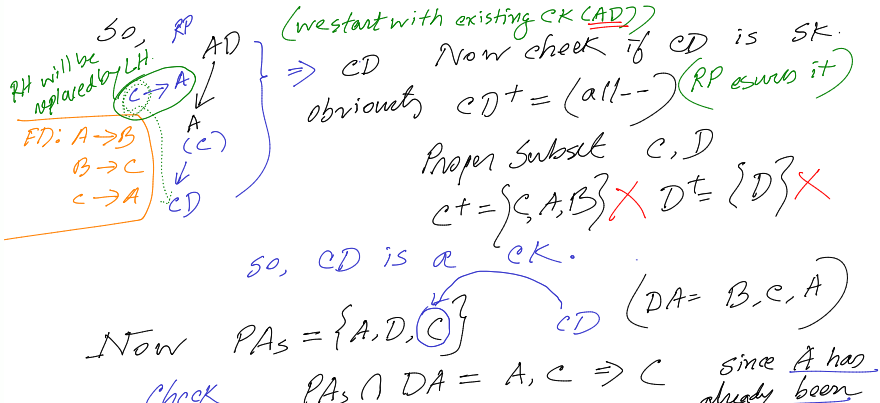
$A \to B$, $B \to C$, $C \to A$ $\Rightarrow DA$

Now, $DA \cap PA_s = A$

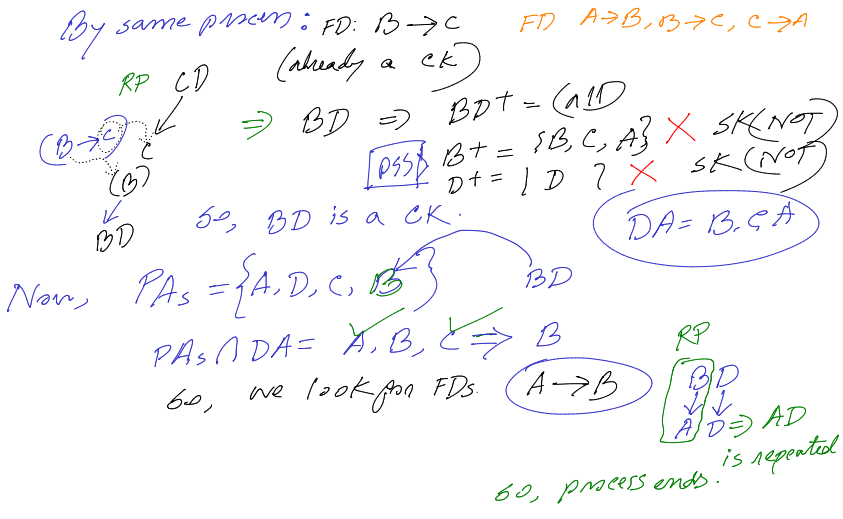
Here, FD: $C \to A$

(Now we look for $\alpha \to A$ then replace A with α, α will replace A)

Method 3: Standard Example Cont.



Method 3: Standard Example Cont.



Method 3: Standard Example (End)

So, Finally we get

set of CKs = {AD, CD, BD}

Summary of Method 3 (which is used)

End of example.

1. Start with all attributes $(A_1 A_2 \cdots A_n)^+$
2. Start discard using given or implied FD
   $|(A_i \cdots A_j)| \leq |A_1 \cdots A_n|$
3. compute $(A_i \cdots A_j)^+ \Rightarrow$ check all proper subsets (final)
4. IF it is a CK then form P.A also form DA = { ? } → updatable

Method 3: Algorithm Review

5. Find $PA \cap DA = x_1 x_2 \cdots x_k$ $k = 1, 2 \ldots$
and we Replacing by $\alpha \rightarrow X$
and go to step 3.

Method 3: More Example A

2 More examples — Example A

Given $R(A,B,C,D)$

FD: $AB \to CD$ $D \to B$ $C \to A$

Solution

all: $ABCD^+ = \{A,B,C,D\}$

discard $ABC\not{D}^+ =$ " $(AB \to CD)$

$AB^+ =$ "

Proper Subsets (PSS)

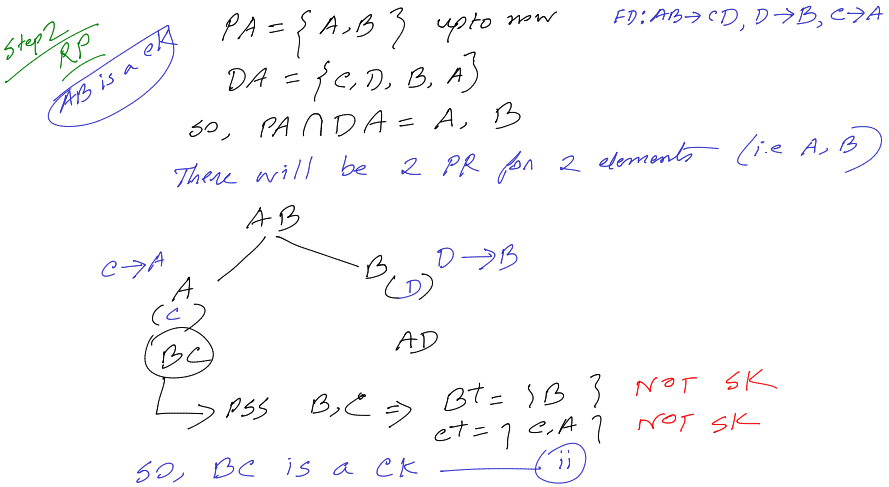
PSS of $AB \Rightarrow A$, B

So, $A^+ = \{A\}$ NOT SK

$B^+ = \{B\}$ NOT SK

So, AB is a CK. ——— (1)

Method 3: More Example A (Cont.)



Method 3: More Example A (Cont.2)

Now, $AD^+ = \{ \text{all} \}$

FD: $AB \to CD$, $D \to B$, $C \to A$

PSS $\begin{cases} A^+ = A \\ D^+ = D, B \end{cases}$

So, AD is a CK (iii)

Now, updated $DA = \{A, B, C, D\}$ BC AD $DA = \{C, D, B, A\}$

$DA \cap DA = \cancel{A, B,}\ C, D = C, D$

None of the FD has C or D in RHS.

So, no RP here.

Method 3: More Example A (End)

But here one more discard can be done that will form another (SK → CK).

FD: AB → CD, D → B, C → A

~~AB~~CD⁺ = all

CD⁺ = "

PSS ⇒ C⁺ = {C, ?} NOT SK

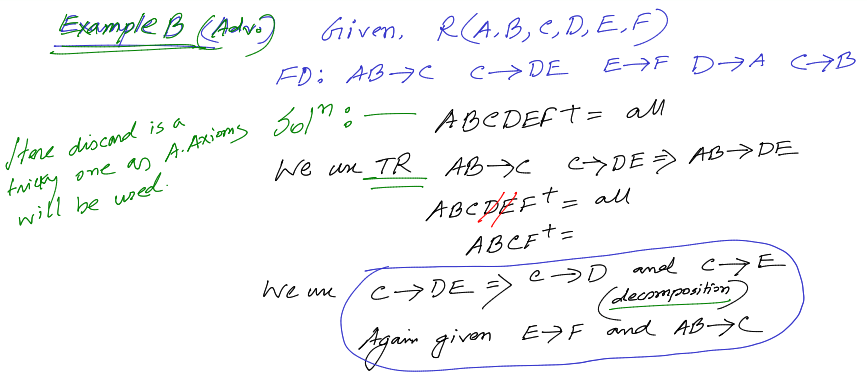
D⁺ = {D, B} NOT SK

So, CD is a CK ——— (iv)

Now, PA = {A, B, C, D} No change, Process ends here.

Ans: {AB, BC, AD, CD} CK

Method 3: More Example B (Begin)



Method 3: More Example B (Cont.)

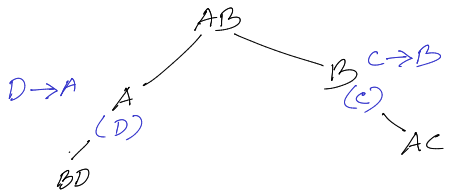
PSS $A^+ = \{A,\}$ NOT SK $B^+ = \{B,\}$ NOT SK

So, AB is a CK.

FD:
- $AB \rightarrow C$
- $C \rightarrow DE$
- $E \rightarrow F$
- $D \rightarrow A$
- $C \rightarrow B$

Now, $PA = \{A, B\}$ $DA = \{C, D, E, F, A, B\}$

$PA \cap DA = A, B$ (Needs 2 PR)



Method 3: More Example B (End)

- BD+ = ___ all

PSS: B+ = ✓ NOT SK
D+ = ✓ NOT SK

FDs: AB → C C → DE
E → F D → A
C → B

So, BD is a CK

- Ac+ = all

PSS: A+ = {A, }
c+ = {c, B, D, E, F, A} = is a SK
also a CK because PSS is have ∅.

So, c is a CK.

Ans. {AB, BD, c}

Normalization: Motivation (Recap)

The **goal** of relational database design is to generate a set of relation schemas that will meet the following 2 goals:

- allows us to store information **without unnecessary redundancy**
- but allows us to **retrieve information easily**
- Hence, we need a standard method to evaluate a design called Normal Forms



Normalization:Motivation Example (Recap)

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

3 Problems:

1. **Redundancy**: Department info is repeated.
2. **Inconsistency**: Update of department info should be propagated properly.
3. **Introduces Bad Business Logic**: You can not enter data for a new department unless there is a teacher of that department.

Atomic Domains and First Normal Form

- A domain is atomic if elements of the domain are considered to be **indivisible units**.
- **Examples**: Hight, weight, country information in general.
- We say that a relation schema **R** is in first normal form (**1NF**) if:
  - The domains of **all attributes of R are atomic**.
  - And the value of each attribute contains **only a single value** from that domain.



First Normal Form: Example

ID	First Name	FamilyName	PhoneNumber
101	a	b	0176543, 0176548
102	c	d	0165438
103	e	f	0176654, 0189756

ID	First Name	FamilyName	PhoneNumber
101	a	b	0176543, 0176548
102	c	d	0165438
103	e	f	0176654, 0189756

- **Multiple values** (even each one is atomic) exist in phone number.

Apparent Solution: Add more attribute

ID	First Name	FamilyName	Phone1	Phone2
101	a	b	0176543	0176548
102	c	d	0165438	
103	e	f	0176654	0189756

- Now it is in 1NF
- But it has **problems**: space wastage (many entries are null).
- Even a larger **design problem**: what if you want to add another phone number? And then another !!!!
- What is the ideal solution?

An Ideal Solution

ID	FirstName	FamilyName
101	a	b
102	c	d
103	e	f

ID	Phone No
101	0176543
101	0176548
102	0165438
103	0176654
103	0189756

- Now it is in 1NF.
- Multiple values can be added efficiently.



1NF: Problem with Integer Values

- Integer is Considered as **atomic** as long as it **has no sub-parts**
- Integer with sub-parts creates problems (e.g. ID= CSE001)
- Extra programming effort is needed to extract dept information and serial number from a given ID.
- Since we need to split the ID to get other information, **the ID is no longer called atomic**.
- So, the schema is NOT in 1NF.
- But if the schema **contains separate attributes** for each of the participating attribute of the ID (in this case: dept) then it is in 1NF. This incurs **redundancy** but is intentional and useful by the end-users.



Second Formal Forms (2NF)

Definition

A relation R is in 2nd Normal Form (2NF) if the following 2 conditions hold:

1. The relation R is in First Normal Form (1NF) (**i.e. all attributes are atomic**)
2. **No Partial Dependency** (P.Dep) exists in the relation R.

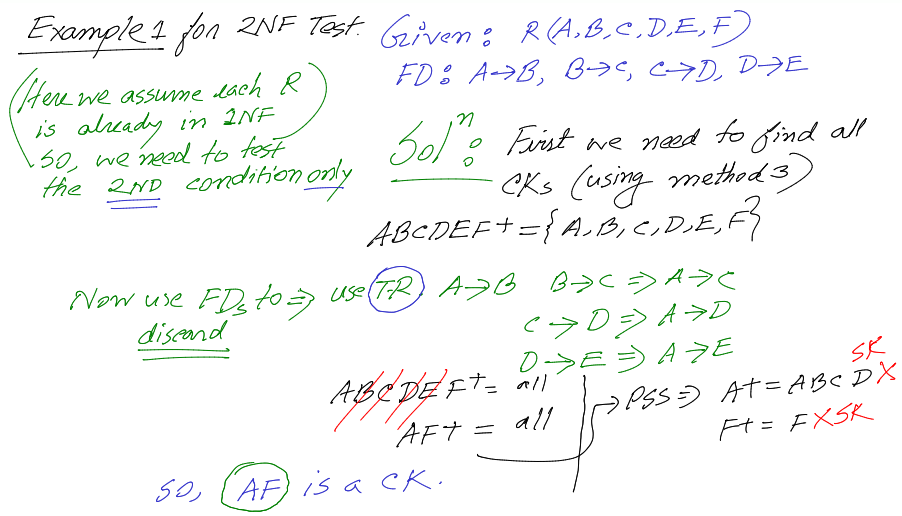
What is Partial Dependency (P.Dep)?

A Relation R is said to have Partial Dependency if

Proper Subset (PSS) of $CK_i \longrightarrow NonPrimeAttribute(NonPA)$

$(NonPA = R - PrimeAttribute)$

Second Formal Forms (2NF) By Example 1



Second Formal Forms (2NF) By Example 1 (Cont.)

Now need to check if more CK exist (i.e $DA \cap PA \neq \emptyset$)

$A \to B \quad B \to C \quad C \to D$

$D \to E$

$R(A, B, C, D, E, F)$

Here, $DA = B, C, D, E$

Upto now $PA = \{A, F\}$

(AF) CK

So, $DA \cap PA = \emptyset$

So, No other CK exists here.

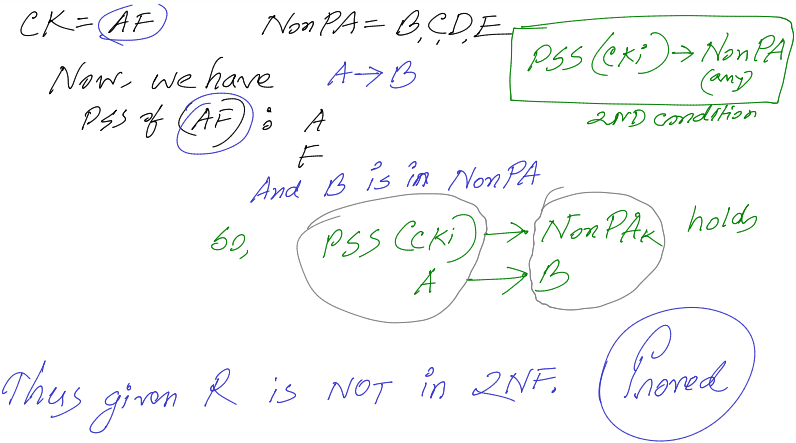
Only CK is (AF) [upto this was required for condition 2]

Condition 2 :

Now CK = AF

Non PA = B, C, D, E (i.e $R - PA$)

Second Formal Forms (2NF) By Example 1 (End)



Second Formal Forms (2NF) By Example 2 (Begin)

2NF Example 2 | Given $R(A, B, C, D)$

FD: $AB \to CD$, $C \to A$, $D \to B$

Soln:

Step1 Find CK (take all) $ABCD^+ = \{A, B, C, D\}$ (RR)

$ABCD^+ =$ " use FD $AB \to CD$ to discard/eliminate

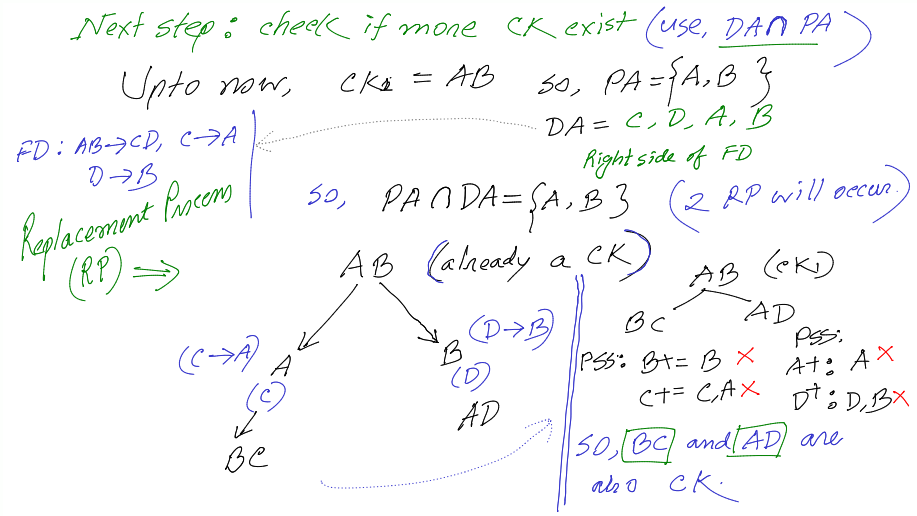
$AB^+ =$ "

PSS of AB: $A^+ = \{A, \}$ SK NO

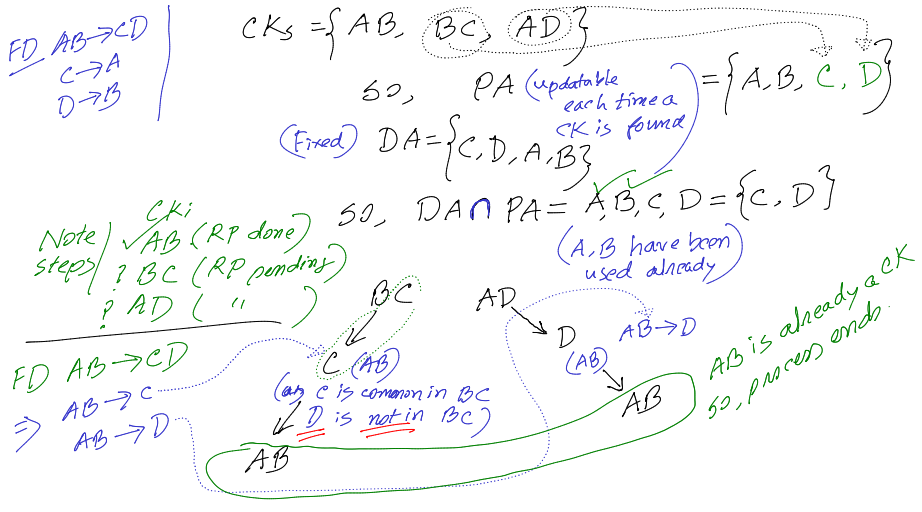
$B^+ = \{B\}$ SK NO

So, AB is a CK

Second Formal Forms (2NF) By Example 2 (Cont.)



Second Formal Forms (2NF) By Example 2 (Cont.2)



Second Formal Forms (2NF) By Example 2 (End)

Step: (upto now we computed all CKs)
Now need to check 2ND condition of 2NF

PSS of $CK_i \rightarrow NonPA_j$

$CKs = AB, BC, AD$ | $PA = \{A, B, C, D\}$
So, $NonPA = R - PA$
$= \{A, B, C, D\} - \{A, B, C, D\}$
$= \emptyset$

Since there is no NonPA the 2ND condition will not hold. So, the Relation is in 2NF.
(Done)

Third Normal Form (3NF)

Motivation

- 2NF states **"No Partial Dependency (P.Dep) exists in the relation R"** to get rid of **Redundancy** (and other associated problems).
- But still anomaly may appear in 2NF due to **NonPrime Transitivity Dependency**. (We will discuss the idea using an example).



Third Normal Form (3NF): One Motivating Example

ID	Name	BG	Division	Country	Weather
101	a	-	Dhaka	BNG	Mild
102	b	-	Dhaka	BNG	Mild
103	c	-	Dhaka	BNG	Mild
104	d	-	Rajshahi	BNG	Hot

Here ID is the SK and CK as well as PK. This Relation is in 2NF (you can check it). But here we observe the following NonPrime Transitivity Dependency:

$Weather \longrightarrow Division, Country$ This **FD introduces Redundancy and update anomaly** in the Relation.

Here, both LHS(Weather) and RHS(Division,Country) are Non Prime Attribute. **NonPrime Transitivity Dependency** will be explained soon.

Third Normal Form (3NF):Conditions

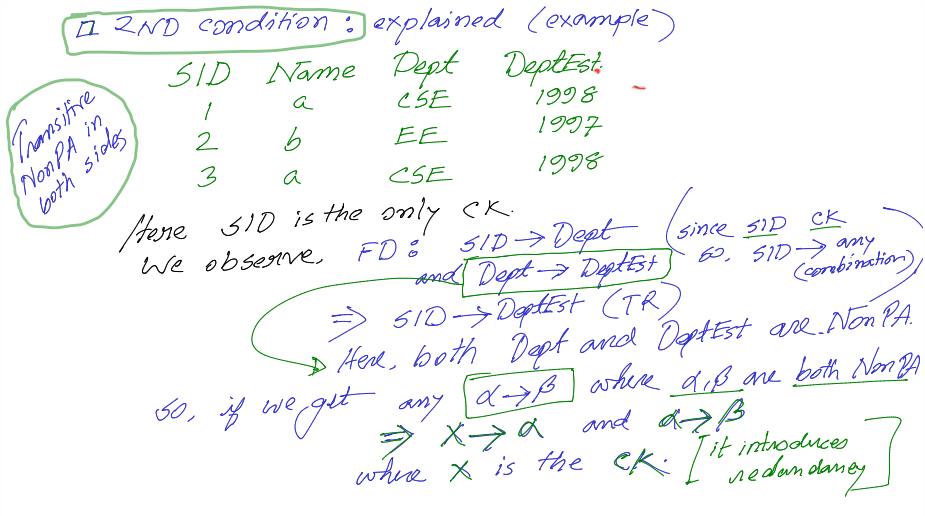
A Relation R is in 3NF if the following **2 conditions** are met:

1. R must be in 2NF (i.e. No Partial Dependency, **PSS of** $(CK_i) \longrightarrow NonPA$)
2. It must **not** contain any **Transitivity Dependency for Non Prime Attributes**, in other words there must not be any ***FD*** : $\alpha \longrightarrow \beta$ where both α and β are Non Prime Attributes.

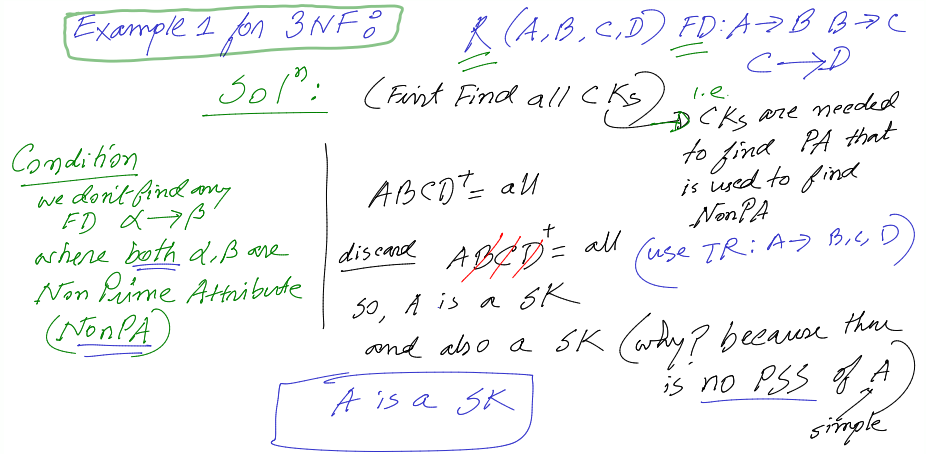
   Note 1: See the conditions are changing gradually from 2NF to 3NF:
   in 2NF we should not have any $PA \longrightarrow NonPA$
   while in 3NF we should not have any $NonPA \longrightarrow NonPA$. It is getting stricter.

   Note 2: Third Normal Form (3NF) is considered adequate for normal relational database design because most of the 3NF tables are free of insertion, update, and deletion anomalies.

3NF Second Condition: Transitivity Non Prime Attribute Dep.



3NF: Example 1 (Begin))



3NF: Example 1 (Cont.)

- Next: is there any more CK?

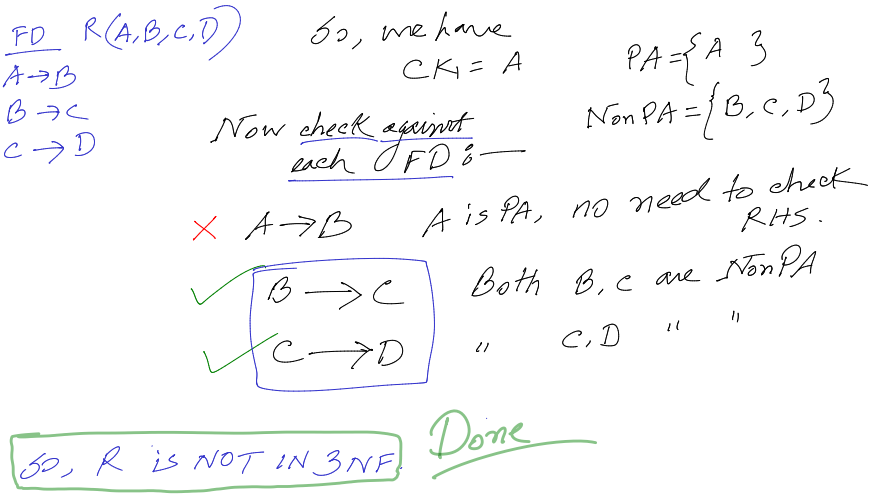
FD:
A → B
B → C
C → D

Upto now, we got, A = CK

So, PA = {A}
and DA = {B, C, D}
∩ = ϕ

So, No other CK possible.

3NF: Example 1 (End))



3NF: Example 2 (Begin))

Example 2 (3NF): Given $R(A,B,C,D,E,F)$

FD: $AB \rightarrow CDEF$, $BD \rightarrow F$

Soln:

Step 1: Find all CKs.

$ABCDEF^+ = (all)$

(discard) $AB\not{C}\not{D}\not{E}\not{F}^+ =$ "

($AB \rightarrow CDEF$)

$AB^+ = (all)$

PSS (AB): $A^+ = A$ ✗ SK

$B^+ = B$ ✗ SK

So, AB is CK

Step 1.2 Any other CK?

CK = AB

PA = A, B

NonPA = C, D, E, F

3NF: Example 2 (End))

upto now, CK = AB, PA = {A, B} DA = CDEF (Right side)

$PA \cap DA = \phi$

So, no CK possible.

Now, apply 2nd condition – CK = AB, PA = A, B

Non PA = C, D, E, F

Check each FD:

AB → CDEF (NO) (Non PA → Non PA exists ?

Exists → BD → F (here, since D is a 'Non Prime BD as a whole is also Non Prime

So, Non PA → Non PA

(i.e. AB → BD & BD → F

⇒ AB → F Transitivity Non PA)

So, R is NOT in 3NF. Done

Boyce - Codd Normal Form (BCNF)

↳ most phenomenal paper in industry in last 30-40 yr. seminal work

Motivation

- BCNF is the advance version of 3NF. It is **stricter** than 3NF and also called **3.5 NF**. BCNF was developed in 1974 by Raymond F. Boyce and Edgar F. Codd to **address certain types of anomalies** not dealt with by 3NF as originally defined.
- **Anomalies** occur in 3NF when we have **Multiple Overlapping Candidate Keys**.



Boyce - Codd Normal Form: Conditions

A Relation R is in BCNF if the following **2 conditions** are met:

1. R must be in 3NF (i.e. No Transitivity Dependency on NonPA (both sides))
2. For **each** Non-trivial Functional Dependency DF:$\alpha \longrightarrow \beta$, α **must be a Superkey (SK)**.

   Note: It is relatively easy to check, for a smaller R you can readily check the RHS and see if it is a SK or not. But here we will use our conventional process we used earlier.

BCNF By Example (Begin)

BCNF Example 1

R(A, B, C)

FD: A → B B → C C → A

Task: To test if R is in BCNF.

Soln:

Note: BCNF removed the problem for multiple overlaping CKs such as AB, BC, CD

Review conditions —
1) Must be in 3NF
2) For all non trivial dep. X → Y
X must be a SK

Note: We'll use same method (3) to get all CKs.
It gives other info needed for BCNF. (i.e SK)

BCNF By Example (Cont.)

$ABC^+ = \{A, B, C\}$

$A \to B$

$C \to A$

$\Rightarrow C^+ = \{all\}$ —— is a SK and CK

(i.e No need to check as No PSS exists)

Again $AB^+ = all$

$B \to C$

$A \to B$

$A^+ = \{all\}$ —— is a SK and CK

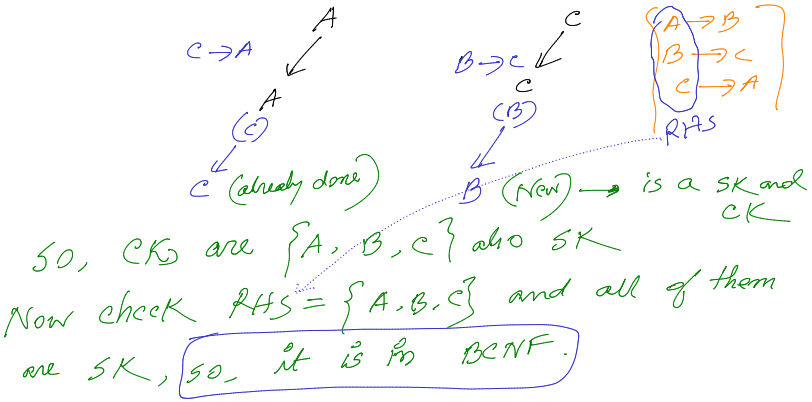
$\Rightarrow$ So, $PA = \{A, C\}$ $DA = A, B, C$

$\alpha = PA \cap DA = A, C$

So, for $CK_1 = C$ $CK_2 = A$

R(ABC)
A→B B→C
C→A

BCNF By Example (End)



2NF: Real Example

Real Example — 2NF

2NF condition:-
No such dependency.
PSS (CKi) → NonPA
Because such FD introduces Redundancy.

Student Course

SID	CID	Credit
1	CSE101	3
2	CSE102	4
1	CSE103	3
3	CSE101	3
2	CSE101	3

Here, CK = (SID, CID) (composite)

Now, we observe. CID → Credit

And CID is a PSS of CK.

So, Right decomposition –

StudentsCourse

(CK) SID	CID (FK)
1	CSE101
2	CSE102

Courses

CID	Credit
CSE101	3
CSE102	4
CSE103	3

3NF: Real Example

feel Example — 3NF

3NF condition

No such Transitivity dependency (TD)

NonPA → NonPA

Students

SID	Dept	Dept Budget
1	CSE	110
2	EEE	120
3	CSE	110

Here we observe — CK = SID

and Dept → DeptBudget

∴ NonPA = Dept, DeptBudget

→ Both Dept and DeptBudget are NonPA.

Explanation of (TD)

SID → Dept and Dept → DeptBudget

⇒ SID → DeptBudget

It means if you have SID and dept then you can deduce deptbudget, so deptbudget in this R is Redundant.

BCNF: Real Example

Real Example — BCNF

Suppose, students can take courses. One course may be conducted by multiple teachers, But one teacher can take only one course.

Students

BCNF condition:

For each Non Trivial FD $\alpha \rightarrow \beta$

α is a SK

SID	CID	TID
1	Java	KH
1	C	SH
2	DB	ARMK
3	Java	HK
4	DB	ANS
4	DB	ARMK
5	C	SH

Here, CK = (SID, CID)

observe — TID → CID

But Non Trivial TID is not a SK.

So, it is not in BCNF

So, we decompose —

SID	TID
1	KH
1	SH
2	ARMK
3	HK
4	ANS
4	ARMK
5	SH

TID	CID
ARMK	DB
KH	Java
SH	C
HK	Java

Thank You

